Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Department of Artificial Intelligence (AI) & Data Science

Name: Aaryan Nandkishor Raorane	**Roll no: S004**
Sap ID: 60018250070	**Topic: Data Structure (Queue)**
Branch: Artificial Intelligence & Data Science	

Experiment No. 3

Aim: To implement Linear Queue and Circular Queue operations in C for a hospital patient registration system, where patients are attended in the order of arrival and emergency counters use circular queues for improved resource utilization.

Theory:

Definition:

A Queue is a linear data structure that follows the First in First out (FIFO) ordering principle. Elements are inserted from one end, called the REAR, and removed from the other end, called the FRONT. The element that enters the queue first is always the first to be removed, reflecting real-world scenarios such as patients registering at a hospital reception counter.

Linear Queue:

In a linear queue implemented using an array, the REAR pointer advances from left to right as elements are inserted. Once REAR reaches the last index (MAX - 1), no further insertions are permitted, even if memory positions at the beginning of the array have been freed by earlier dequeue operations. This results in a condition known as false overflow.

Circular Queue:

A circular queue overcomes the false overflow problem by logically connecting the last position of the array back to the first, forming a ring structure. Both FRONT and REAR pointers wrap around using modular arithmetic (rear = (rear + 1) % MAX), allowing freed slots to be reused without shifting elements.

Operations:

Linear Queue operations:

a) Enqueue:

It is used to insert a new patient name at the REAR of the queue. If the queue is full (rear == MAX - 1), insertion is rejected. On the first enqueue, FRONT is initialized to 0.

Pseudocode:

```
BEGIN ENQUEUE(name)
  IF rear == MAX - 1 THEN
    PRINT "Queue is Full! Cannot register more patients."
  ELSE
    IF front == -1 THEN
      front = 0
    END IF
    rear = rear + 1
    queue[rear] = name
    PRINT "Patient registered successfully."
  END IF
END ENQUEUE
```

b) Dequeue:

It is used to remove and serve the patient at the FRONT of the queue. If the queue is empty (front == -1), the operation is rejected. When the last element is removed (front == rear), both pointers are reset to -1.

Pseudocode:

```
BEGIN DEQUEUE()
  IF front == -1 THEN
    PRINT "Queue is Empty! No patients to attend."
  ELSE
    PRINT "Patient " + queue[front] + " is being attended."
    IF front == rear THEN
      front = -1
      rear  = -1
```

```
        ELSE
            front = front + 1
        END IF
    END IF
END DEQUEUE
```

c) Display:

It traverses the array from index front to rear and prints each patient name with a serial number.

Pseudocode:

```
BEGIN DISPLAY()
    IF front == -1 THEN
        PRINT "No patients waiting in queue."
    ELSE
        PRINT "Patients Waiting in Queue:"
        SET i = front
        WHILE i <= rear DO
            PRINT queue[i]
            i = i + 1
        END WHILE
    END IF
END DISPLAY
```

d) IsEmpty:

It returns 1 if front equals -1, indicating no patients are present in the queue.

e) IsFull:

It returns 1 if rear equals MAX - 1, indicating no further registrations are possible in the linear queue.

Circular Queue Operations:

a) Enqueue:

It is used to insert a new emergency patient name. The full condition is checked as (rear + 1) % MAX == front. REAR advances as (rear + 1) % MAX, wrapping around to reuse freed positions.

Pseudocode:

```
BEGIN CENQUEUE(name)
  IF (rear + 1) % MAX == front THEN
    PRINT "Emergency Counter Full!"
  ELSE
    IF front == -1 THEN
      front = 0
      rear  = 0
    ELSE
      rear = (rear + 1) % MAX
    END IF
    cqueue[rear] = name
    PRINT "Emergency Patient registered."
  END IF
END CENQUEUE
```

b) Dequeue:

It removes and serves the emergency patient at FRONT. If front equals rear after removal, both pointers are reset to -1; otherwise FRONT advances as (front + 1) % MAX.

Pseudocode:

```
BEGIN CDEQUEUE()
  IF front == -1 THEN
    PRINT "No emergency patients in queue."
  ELSE
    PRINT "Attending: " + cqueue[front]
    IF front == rear THEN
      front = -1
      rear  = -1
    ELSE
      front = (front + 1) % MAX
    END IF
  END IF
```

```
END CDEQUEUE
```

c) Display:

It starts at index front and iterates using (i + 1) % MAX, printing each emergency patient name until rear is reached.

Pseudocode:

```
BEGIN CDISPLAY()
  IF front == -1 THEN
    PRINT "No patients in emergency queue."
  ELSE
    PRINT "Emergency Patients in Queue:"
    SET i = front
    WHILE 1 DO
      PRINT cqueue[i]
      IF i == rear THEN
        BREAK
      END IF
      i = (i + 1) % MAX
    END WHILE
  END IF
END CDISPLAY
```

d) IsEmpty:

It returns 1 if front equals -1, indicating the emergency queue is empty.

e) IsFull:

It returns 1 if (rear + 1) % MAX equals front, indicating all circular slots are occupied.

Code:

Program 1: Linear Queue — Hospital Patient Registration

```
#include<stdio.h>
#include<conio.h>
#include<string.h>

#define MAX 5

char queue[MAX][50];
int front = -1, rear = -1;

int isFull()
{
   if(rear == MAX - 1)
      return 1;
   return 0;
}

int isEmpty()
{
   if(front == -1)
      return 1;
   return 0;
}

void enqueue()
{
   char name[50];
   if(isFull())
   {
      printf("\nQueue is Full! Cannot register more patients.");
      return;
   }
   if(front == -1)
      front = 0;
   rear++;
   printf("\nEnter Patient Name: ");
   gets(name);
   strcpy(queue[rear], name);
   printf("\nPatient '%s' registered successfully.", name);
}

void dequeue()
{
   if(isEmpty())
   {
      printf("\nQueue is Empty! No patients to attend.");
      return;
   }
   printf("\nPatient '%s' is being attended.", queue[front]);
   if(front == rear)
   {
```

```
        front = -1;
        rear  = -1;
    }
    else
        front++;
}

void display()
{
    int i;
    if(isEmpty())
    {
        printf("\nNo patients waiting in queue.");
        return;
    }
    printf("\nPatients Waiting in Queue:\n");
    for(i = front; i <= rear; i++)
        printf("%d. %s\n", i - front + 1, queue[i]);
}

void main()
{
    int choice;
    clrscr();
    do
    {
        printf("\n\n--- Hospital Patient Registration (Linear Queue) ---");
        printf("\n1. Register Patient (Enqueue)");
        printf("\n2. Attend Patient  (Dequeue)");
        printf("\n3. Display Queue");
        printf("\n4. Exit");
        printf("\nEnter Your Choice: ");
        scanf("%d", &choice);
        fflush(stdin);
        switch(choice)
        {
            case 1: enqueue(); break;
            case 2: dequeue(); break;
            case 3: display(); break;
            case 4: printf("\nExiting System..."); break;
            default: printf("\nInvalid Choice! Try Again.");
        }
    }while(choice != 4);
    getch();
}
```

Output:

```
--- Hospital Patient Registration (Linear Queue) ---
1. Register Patient (Enqueue)
```

```
2. Attend Patient  (Dequeue)
3. Display Queue
4. Exit
Enter Your Choice: 1

Enter Patient Name: Aarav Sharma

Patient 'Aarav Sharma' registered successfully.


--- Hospital Patient Registration (Linear Queue) ---
1. Register Patient (Enqueue)
2. Attend Patient  (Dequeue)
3. Display Queue
4. Exit
Enter Your Choice: 1

Enter Patient Name: Priya Mehta

Patient 'Priya Mehta' registered successfully.


--- Hospital Patient Registration (Linear Queue) ---
1. Register Patient (Enqueue)
2. Attend Patient  (Dequeue)
3. Display Queue
4. Exit
Enter Your Choice: 1

Enter Patient Name: Rohan Kapoor

Patient 'Rohan Kapoor' registered successfully.


--- Hospital Patient Registration (Linear Queue) ---
1. Register Patient (Enqueue)
2. Attend Patient  (Dequeue)
3. Display Queue
4. Exit
Enter Your Choice: 3

Patients Waiting in Queue:
```

```
1. Aarav Sharma
2. Priya Mehta
3. Rohan Kapoor


--- Hospital Patient Registration (Linear Queue) ---
1. Register Patient (Enqueue)
2. Attend Patient  (Dequeue)
3. Display Queue
4. Exit
Enter Your Choice: 2

Patient 'Aarav Sharma' is being attended.


--- Hospital Patient Registration (Linear Queue) ---
1. Register Patient (Enqueue)
2. Attend Patient  (Dequeue)
3. Display Queue
4. Exit
Enter Your Choice: 1

Enter Patient Name: Sneha Iyer

Patient 'Sneha Iyer' registered successfully.


--- Hospital Patient Registration (Linear Queue) ---
1. Register Patient (Enqueue)
2. Attend Patient  (Dequeue)
3. Display Queue
4. Exit
Enter Your Choice: 1

Enter Patient Name: Vikram Rao

Patient 'Vikram Rao' registered successfully.


--- Hospital Patient Registration (Linear Queue) ---
1. Register Patient (Enqueue)
2. Attend Patient  (Dequeue)
```

```
3. Display Queue
4. Exit
Enter Your Choice: 1

Queue is Full! Cannot register more patients.


--- Hospital Patient Registration (Linear Queue) ---
1. Register Patient (Enqueue)
2. Attend Patient  (Dequeue)
3. Display Queue
4. Exit
Enter Your Choice: 2

Patient 'Priya Mehta' is being attended.


--- Hospital Patient Registration (Linear Queue) ---
1. Register Patient (Enqueue)
2. Attend Patient  (Dequeue)
3. Display Queue
4. Exit
Enter Your Choice: 2

Patient 'Rohan Kapoor' is being attended.


--- Hospital Patient Registration (Linear Queue) ---
1. Register Patient (Enqueue)
2. Attend Patient  (Dequeue)
3. Display Queue
4. Exit
Enter Your Choice: 2

Patient 'Sneha Iyer' is being attended.


--- Hospital Patient Registration (Linear Queue) ---
1. Register Patient (Enqueue)
2. Attend Patient  (Dequeue)
3. Display Queue
4. Exit
```

```
Enter Your Choice: 3

Patients Waiting in Queue:
1. Vikram Rao


--- Hospital Patient Registration (Linear Queue) ---
1. Register Patient (Enqueue)
2. Attend Patient  (Dequeue)
3. Display Queue
4. Exit
Enter Your Choice: 2

Patient 'Vikram Rao' is being attended.


--- Hospital Patient Registration (Linear Queue) ---
1. Register Patient (Enqueue)
2. Attend Patient  (Dequeue)
3. Display Queue
4. Exit
Enter Your Choice:

3

No patients waiting in queue.


--- Hospital Patient Registration (Linear Queue) ---
1. Register Patient (Enqueue)
2. Attend Patient  (Dequeue)
3. Display Queue
4. Exit
Enter Your Choice: 4

Exiting System...


=== Code Execution Successful ===
```

Program 2: Circular Queue — Hospital Emergency Counter

```
#include<stdio.h>
#include<conio.h>
#include<string.h>

#define MAX 5

char cqueue[MAX][50];
int front = -1, rear = -1;

int isFull()
{
   if((rear + 1) % MAX == front)
      return 1;
   return 0;
}

int isEmpty()
{
   if(front == -1)
      return 1;
   return 0;
}

void enqueue()
{
   char name[50];
   if(isFull())
   {
      printf("\nEmergency Counter Full! Cannot add more patients.");
      return;
   }
   if(front == -1)
   {
      front = 0;
      rear  = 0;
   }
   else
      rear = (rear + 1) % MAX;
   printf("\nEnter Emergency Patient Name: ");
   gets(name);
   strcpy(cqueue[rear], name);
   printf("\nEmergency Patient '%s' registered.", name);
}

void dequeue()
{
   if(isEmpty())
   {
      printf("\nNo emergency patients in queue.");
      return;
   }
```

```
    printf("\nAttending Emergency Patient: %s", cqueue[front]);
    if(front == rear)
    {
        front = -1;
        rear  = -1;
    }
    else
        front = (front + 1) % MAX;
}

void display()
{
    int i;
    if(isEmpty())
    {
        printf("\nNo patients in emergency queue.");
        return;
    }
    printf("\nEmergency Patients in Queue:\n");
    i = front;
    while(1)
    {
        printf("- %s\n", cqueue[i]);
        if(i == rear)
            break;
        i = (i + 1) % MAX;
    }
}

void main()
{
    int choice;
    clrscr();
    do
    {
        printf("\n\n--- Hospital Emergency Counter (Circular Queue) ---");
        printf("\n1. Register Emergency Patient (Enqueue)");
        printf("\n2. Attend Emergency Patient  (Dequeue)");
        printf("\n3. Display Emergency Queue");
        printf("\n4. Exit");
        printf("\nEnter Your Choice: ");
        scanf("%d", &choice);
        fflush(stdin);
        switch(choice)
        {
            case 1: enqueue(); break;
            case 2: dequeue(); break;
            case 3: display(); break;
            case 4: printf("\nExiting System..."); break;
            default: printf("\nInvalid Choice! Try Again.");
        }
    }while(choice != 4);
    getch();
}
```

OUTPUT:

```
--- Hospital Emergency Counter (Circular Queue) ---
1. Register Emergency Patient (Enqueue)
2. Attend Emergency Patient (Dequeue)
3. Display Emergency Queue
4. Exit
Enter Your Choice: 1

Enter Emergency Patient Name: Aarav Sharma

Emergency Patient 'Aarav Sharma' registered.


--- Hospital Emergency Counter (Circular Queue) ---
1. Register Emergency Patient (Enqueue)
2. Attend Emergency Patient (Dequeue)
3. Display Emergency Queue
4. Exit
Enter Your Choice: 1

Enter Emergency Patient Name: Priya Mehta

Emergency Patient 'Priya Mehta' registered.


--- Hospital Emergency Counter (Circular Queue) ---
1. Register Emergency Patient (Enqueue)
2. Attend Emergency Patient (Dequeue)
3. Display Emergency Queue
4. Exit
Enter Your Choice: 1

Enter Emergency Patient Name: Rohan Kapoor

Emergency Patient 'Rohan Kapoor' registered.


--- Hospital Emergency Counter (Circular Queue) ---
```

```
1. Register Emergency Patient (Enqueue)
2. Attend Emergency Patient (Dequeue)
3. Display Emergency Queue
4. Exit
Enter Your Choice: 1

Enter Emergency Patient Name: Sneha Iyer

Emergency Patient 'Sneha Iyer' registered.


--- Hospital Emergency Counter (Circular Queue) ---
1. Register Emergency Patient (Enqueue)
2. Attend Emergency Patient (Dequeue)
3. Display Emergency Queue
4. Exit
Enter Your Choice: 1

Enter Emergency Patient Name: Vikram Rao

Emergency Patient 'Vikram Rao' registered.


--- Hospital Emergency Counter (Circular Queue) ---
1. Register Emergency Patient (Enqueue)
2. Attend Emergency Patient (Dequeue)
3. Display Emergency Queue
4. Exit
Enter Your Choice: 1

Emergency Counter Full! Cannot add more patients.


--- Hospital Emergency Counter (Circular Queue) ---
1. Register Emergency Patient (Enqueue)
2. Attend Emergency Patient (Dequeue)
3. Display Emergency Queue
4. Exit
Enter Your Choice: 3

Emergency Patients in Queue:
- Aarav Sharma
```

```
- Priya Mehta
- Rohan Kapoor
- Sneha Iyer
- Vikram Rao


--- Hospital Emergency Counter (Circular Queue) ---
1. Register Emergency Patient (Enqueue)
2. Attend Emergency Patient (Dequeue)
3. Display Emergency Queue
4. Exit
Enter Your Choice: 2

Attending Emergency Patient: Aarav Sharma


--- Hospital Emergency Counter (Circular Queue) ---
1. Register Emergency Patient (Enqueue)
2. Attend Emergency Patient (Dequeue)
3. Display Emergency Queue
4. Exit
Enter Your Choice: 2

Attending Emergency Patient: Priya Mehta


--- Hospital Emergency Counter (Circular Queue) ---
1. Register Emergency Patient (Enqueue)
2. Attend Emergency Patient (Dequeue)
3. Display Emergency Queue
4. Exit
Enter Your Choice: 2

Attending Emergency Patient: Rohan Kapoor


--- Hospital Emergency Counter (Circular Queue) ---
1. Register Emergency Patient (Enqueue)
2. Attend Emergency Patient (Dequeue)
3. Display Emergency Queue
4. Exit
Enter Your Choice: 1
```

```
Enter Emergency Patient Name: Anaya Mishra

Emergency Patient 'Anaya Mishra' registered.


--- Hospital Emergency Counter (Circular Queue) ---
1. Register Emergency Patient (Enqueue)
2. Attend Emergency Patient (Dequeue)
3. Display Emergency Queue
4. Exit
Enter Your Choice: 3

Emergency Patients in Queue:
- Sneha Iyer
- Vikram Rao
- Anaya Mishra


--- Hospital Emergency Counter (Circular Queue) ---
1. Register Emergency Patient (Enqueue)
2. Attend Emergency Patient (Dequeue)
3. Display Emergency Queue
4. Exit
Enter Your Choice: 2

Attending Emergency Patient: Sneha Iyer


--- Hospital Emergency Counter (Circular Queue) ---
1. Register Emergency Patient (Enqueue)
2. Attend Emergency Patient (Dequeue)
3. Display Emergency Queue
4. Exit
Enter Your Choice: 2

Attending Emergency Patient: Vikram Rao


--- Hospital Emergency Counter (Circular Queue) ---
1. Register Emergency Patient (Enqueue)
2. Attend Emergency Patient (Dequeue)
```

```
3. Display Emergency Queue
4. Exit
Enter Your Choice: 2

Attending Emergency Patient: Anaya Mishra


--- Hospital Emergency Counter (Circular Queue) ---
1. Register Emergency Patient (Enqueue)
2. Attend Emergency Patient (Dequeue)
3. Display Emergency Queue
4. Exit
Enter Your Choice: 2

No emergency patients in queue.


--- Hospital Emergency Counter (Circular Queue) ---
1. Register Emergency Patient (Enqueue)
2. Attend Emergency Patient (Dequeue)
3. Display Emergency Queue
4. Exit
Enter Your Choice: 5

Invalid Choice! Try Again.


--- Hospital Emergency Counter (Circular Queue) ---
1. Register Emergency Patient (Enqueue)
2. Attend Emergency Patient (Dequeue)
3. Display Emergency Queue
4. Exit
Enter Your Choice: 4

Exiting System...


=== Code Execution Successful ===
```

Applications:

- Patient registration at hospital reception desks, where patients are served strictly in the order of their arrival.
- CPU process scheduling using the Round Robin algorithm, where processes are placed in a circular queue and each is allocated an equal time slice.
- Traffic signal management systems that cycle through signal states at intersections in a fixed repeating order.
- Print spooler systems that receive and execute print jobs in submission order.
- Network packet buffering in routers, where incoming data packets are queued for ordered transmission.
- Keyboard input buffering in operating systems, where keystrokes are stored in a queue until the application processes them.

Conclusion:

Both linear and circular queue data structures successfully follow the FIFO principle and provide systematic management of patient data in a hospital setting. The linear queue offers a straightforward implementation where patients are registered at the REAR and attended from the FRONT, with O(1) time complexity for all enqueue and dequeue operations. However, it suffers from false overflow, where the REAR reaches the end of the array even when positions at the front are vacant. The circular queue, used for the emergency counter, eliminates this limitation entirely by wrapping both pointers using modular arithmetic, enabling reuse of all available memory slots without any element shifting. This assignment demonstrated the practical significance of choosing the appropriate queue variant based on the constraints and throughput requirements of a given system.